Binance Gold & Silver Perpetuals and ETF Markets

KSE Python course project

1. Introduction and research question

Research question.
Did the introduction of Binance TRADFI gold and silver perpetual futures change volatility, liquidity and price integration in the corresponding precious metal ETF markets (GLD and SLV)?

Main hypotheses.

- **H1 (volatility)**: Realized volatility of gold and silver ETFs increased after the launch of Binance XAUUSDT and XAGUSDT TRADFI perpetual futures.
- **H2 (liquidity)**: Trading activity and liquidity of GLD and SLV improved (higher volume, lower Amihud illiquidity) after the launch.
- **H3 (integration)**: Short-run price co-movement and beta of GLD/SLV with Binance XAUUSDT/XAGUSDT returns increased after the launch.

2. Data and feature set

This section loads the pre-built daily feature dataset `data/processed/features_daily.csv`. Raw data sources and feature definitions are described in `DATA_DESCRIPTION.md` and `VARIABLES_DESCRIPTION.md`.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

plt.style.use("seaborn-v0_8")

# Load features (note: the date column is named 'Date' in the CSV)
df = pd.read_csv("data/processed/features_daily.csv", parse_dates=["Date"])
df = df.set_index("Date").sort_index()

df.shape
```

```
(112, 85)
```

```python
df.head()
```

Date	px_gld_open	px_gld_high	px_gld_low	px_gld_close	vol_gld	px_slv_open	px_
2025-09-02	320.820007	325.920013	320.239990	325.589996	21247500.0	36.630001	37.
2025-09-03	327.760010	329.450012	326.730011	328.140015	16062200.0	37.330002	37.
2025-09-04	327.029999	327.559998	325.350006	326.690002	11703100.0	37.200001	37.
2025-09-05	329.529999	331.440002	328.929993	331.049988	16065200.0	37.410000	37.
2025-09-08	333.660004	335.670013	333.230011	334.820007	18617300.0	37.570000	37.

2.1 Basic cleaning and event date

We define the main event date as the launch of Binance XAUUSDT TRADFI perpetual futures (2026-01-05).
To keep a meaningful pre-event window, we only require that ETF prices and returns are available; Binance-specific fields are allowed to be missing before the launch.

```python
event_date = pd.Timestamp("2026-01-05")

# Require only ETF prices/returns (pre-event we do not have Binance data)
core_cols = ["px_gld_close", "px_slv_close", "ret_gld", "ret_slv"]
df_core = df.dropna(subset=core_cols).copy()

df_core["post_event"] = (df_core.index >= event_date).astype(int)

df_core.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 111 entries, 2025-09-03 to 2026-02-10
Data columns (total 85 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   px_gld_open                     111 non-null    float64
 1   px_gld_high                     111 non-null    float64
 2   px_gld_low                      111 non-null    float64
 3   px_gld_close                    111 non-null    float64
 4   vol_gld                         111 non-null    float64
 5   px_slv_open                     111 non-null    float64
 6   px_slv_high                     111 non-null    float64
 7   px_slv_low                      111 non-null    float64
 8   px_slv_close                    111 non-null    float64
 9   vol_slv                         111 non-null    float64
 10  dxy                             111 non-null    float64
```

```
11  vix                              111 non-null    float64
12  tnx                              111 non-null    float64
13  px_xau_binance_open              26 non-null     float64
14  px_xau_binance_high              26 non-null     float64
15  px_xau_binance_low               26 non-null     float64
16  px_xau_binance_close             26 non-null     float64
17  px_xau_binance_volume            26 non-null     float64
18  px_xau_binance_num_trades        26 non-null     float64
19  px_xag_binance_open              23 non-null     float64
20  px_xag_binance_high              23 non-null     float64
21  px_xag_binance_low               23 non-null     float64
22  px_xag_binance_close             23 non-null     float64
23  px_xag_binance_volume            23 non-null     float64
24  px_xag_binance_num_trades        23 non-null     float64
25  funding_xau                      22 non-null     float64
26  funding_mark_xau                 22 non-null     float64
27  funding_xag                      22 non-null     float64
28  funding_mark_xag                 22 non-null     float64
29  oi_xau                           0 non-null      float64
30  oi_xag                           0 non-null      float64
31  ret_xau_binance                  25 non-null     float64
32  ret_xag_binance                  22 non-null     float64
33  ret_gld                          111 non-null    float64
34  ret_slv                          111 non-null    float64
35  ret_xau_binance_abs              25 non-null     float64
36  ret_xau_binance_sq               25 non-null     float64
37  ret_xag_binance_abs              22 non-null     float64
38  ret_xag_binance_sq               22 non-null     float64
39  ret_gld_abs                      111 non-null    float64
40  ret_gld_sq                       111 non-null    float64
41  ret_slv_abs                      111 non-null    float64
42  ret_slv_sq                       111 non-null    float64
43  vol_realized_10_ret_xau_binance  16 non-null     float64
44  vol_realized_20_ret_xau_binance  6 non-null      float64
45  vol_realized_10_ret_xag_binance  13 non-null     float64
46  vol_realized_20_ret_xag_binance  3 non-null      float64
47  vol_realized_10_ret_gld          102 non-null    float64
48  vol_realized_20_ret_gld          92 non-null     float64
49  vol_realized_10_ret_slv          102 non-null    float64
50  vol_realized_20_ret_slv          92 non-null     float64
51  vol_parkinson_gld                93 non-null     float64
52  vol_parkinson_slv                93 non-null     float64
53  vol_parkinson_xau_binance        7 non-null      float64
54  vol_parkinson_xag_binance        4 non-null      float64
55  vol_gk_gld                       93 non-null     float64
56  vol_gk_slv                       93 non-null     float64
```

```
 57  vol_gk_xau_binance              7 non-null      float64
 58  vol_gk_xag_binance              4 non-null      float64
 59  corr_20_gld_xau                 6 non-null      float64
 60  corr_20_slv_xag                 3 non-null      float64
 61  beta_60_gld_on_xau              0 non-null      float64
 62  beta_60_slv_on_xag              0 non-null      float64
 63  vol_ratio_gld_xau               6 non-null      float64
 64  vol_ratio_slv_xag               3 non-null      float64
 65  abs_return_gap_gld_xau          25 non-null     float64
 66  abs_return_gap_slv_xag          22 non-null     float64
 67  post_event                      111 non-null    int64
 68  event_window_7                  111 non-null    int64
 69  event_window_14                 111 non-null    int64
 70  log_vol_gld                     111 non-null    float64
 71  log_vol_slv                     111 non-null    float64
 72  amihud_gld                      111 non-null    float64
 73  amihud_20_gld                   92 non-null     float64
 74  amihud_slv                      111 non-null    float64
 75  amihud_20_slv                   92 non-null     float64
 76  funding_xau_change              21 non-null     float64
 77  funding_extreme_xau             111 non-null    int64
 78  oi_xau_change                   0 non-null      float64
 79  funding_xag_change              21 non-null     float64
 80  funding_extreme_xag             111 non-null    int64
 81  oi_xag_change                   0 non-null      float64
 82  dxy_ret                         111 non-null    float64
 83  vix_ret                         111 non-null    float64
 84  tnx_ret                         111 non-null    float64
dtypes: float64(80), int64(5)
memory usage: 74.6 KB
```

```python
df_core[["post_event"]].value_counts().sort_index()
```

```
post_event
0             85
1             26
Name: count, dtype: int64
```

3. Descriptive statistics

3.1 Prices and returns

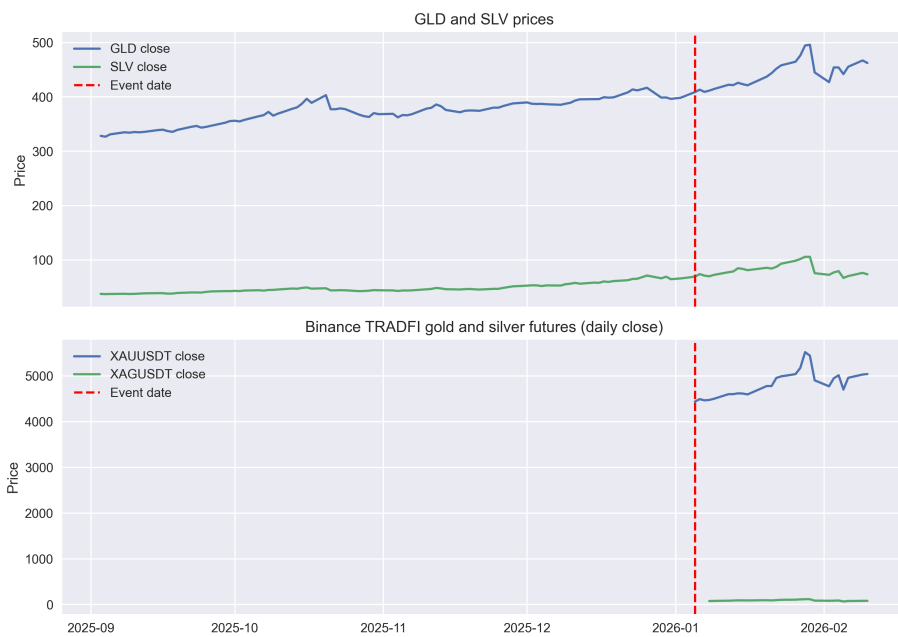
```python
fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True)

# ETF prices
axes[0].plot(df_core.index, df_core["px_gld_close"], label="GLD close")
```

```
axes[0].plot(df_core.index, df_core["px_slv_close"], label="SLV close")
axes[0].axvline(event_date, color="red", linestyle="--", label="Event date")
axes[0].set_ylabel("Price")
axes[0].set_title("GLD and SLV prices")
axes[0].legend()

# Binance aggregated close prices
axes[1].plot(df_core.index, df_core["px_xau_binance_close"], label="XAUUSDT close")
axes[1].plot(df_core.index, df_core["px_xag_binance_close"], label="XAGUSDT close")
axes[1].axvline(event_date, color="red", linestyle="--", label="Event date")
axes[1].set_ylabel("Price")
axes[1].set_title("Binance TRADFI gold and silver futures (daily close)")
axes[1].legend()

plt.tight_layout()
plt.show()
```




```
returns_cols = ["ret_gld", "ret_slv", "ret_xau_binance", "ret_xag_binance"]
df_core[returns_cols].describe().T[["mean", "std", "min", "max"]]
```

	mean	std	min	max
ret_gld	0.003160	0.019440	-0.108412	0.061647
ret_slv	0.006136	0.048138	-0.336037	0.086602
ret_xau_binance	0.005092	0.034425	-0.104457	0.064529

	mean	std	min	max
ret_xag_binance	0.002603	0.106036	-0.312222	0.145629

3.2 Volatility and liquidity over time

We look at 20-day realized volatility for GLD and SLV and ETF trading volume.

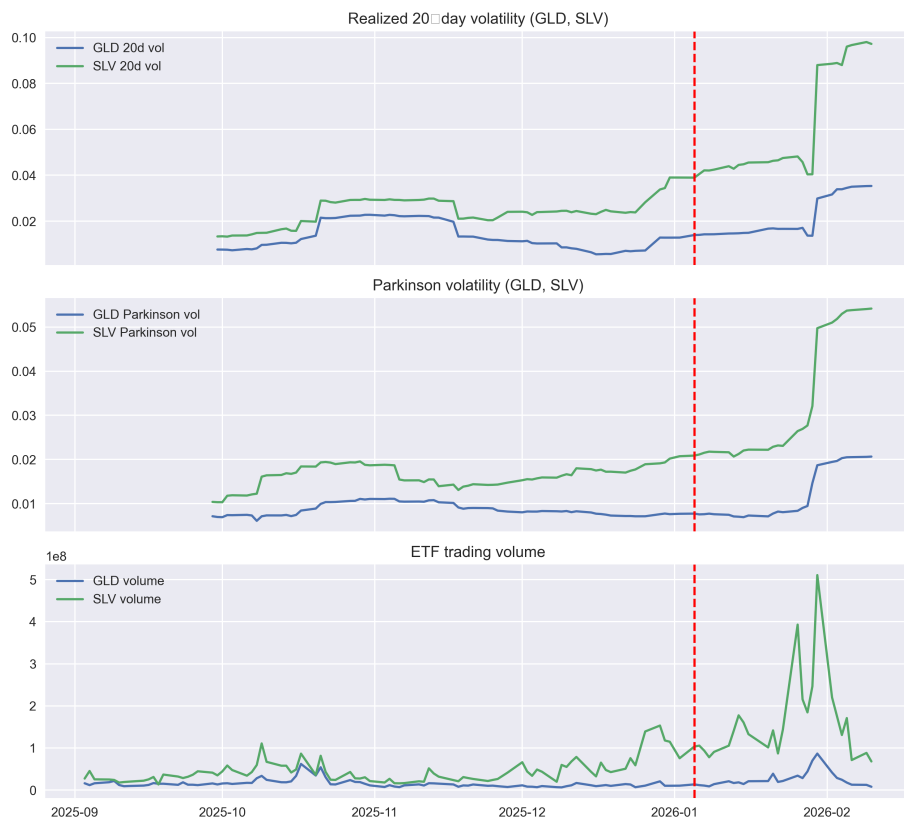
```
fig, axes = plt.subplots(3, 1, figsize=(10, 9), sharex=True)

axes[0].plot(df_core.index, df_core["vol_realized_20_ret_gld"], label="GLD 20d vol")
axes[0].plot(df_core.index, df_core["vol_realized_20_ret_slv"], label="SLV 20d vol")
axes[0].axvline(event_date, color="red", linestyle="--")
axes[0].set_title("Realized 20-day volatility (GLD, SLV)")
axes[0].legend()

axes[1].plot(df_core.index, df_core["vol_parkinson_gld"], label="GLD Parkinson vol")
axes[1].plot(df_core.index, df_core["vol_parkinson_slv"], label="SLV Parkinson vol")
axes[1].axvline(event_date, color="red", linestyle="--")
axes[1].set_title("Parkinson volatility (GLD, SLV)")
axes[1].legend()

if "vol_gld" in df_core.columns and "vol_slv" in df_core.columns:
    axes[2].plot(df_core.index, df_core["vol_gld"], label="GLD volume")
    axes[2].plot(df_core.index, df_core["vol_slv"], label="SLV volume")
    axes[2].axvline(event_date, color="red", linestyle="--")
    axes[2].set_title("ETF trading volume")
    axes[2].legend()

plt.tight_layout()
plt.show()
```




```
liq_cols = []
for c in ["amihud_gld", "amihud_20_gld", "amihud_slv", "amihud_20_slv"]:
    if c in df_core.columns:
        liq_cols.append(c)

df_core[liq_cols].describe().T[["mean", "std", "min", "max"]]
```

	mean	std	min	max
amihud_gld	6.977909e-10	5.506247e-10	3.474315e-12	2.390221e-09
amihud_20_gld	6.709650e-10	1.040833e-10	4.831240e-10	9.406983e-10
amihud_slv	4.710888e-10	3.304889e-10	1.153311e-12	2.008110e-09
amihud_20_slv	4.803576e-10	1.172749e-10	2.962718e-10	7.452803e-10

3.3 Correlogram

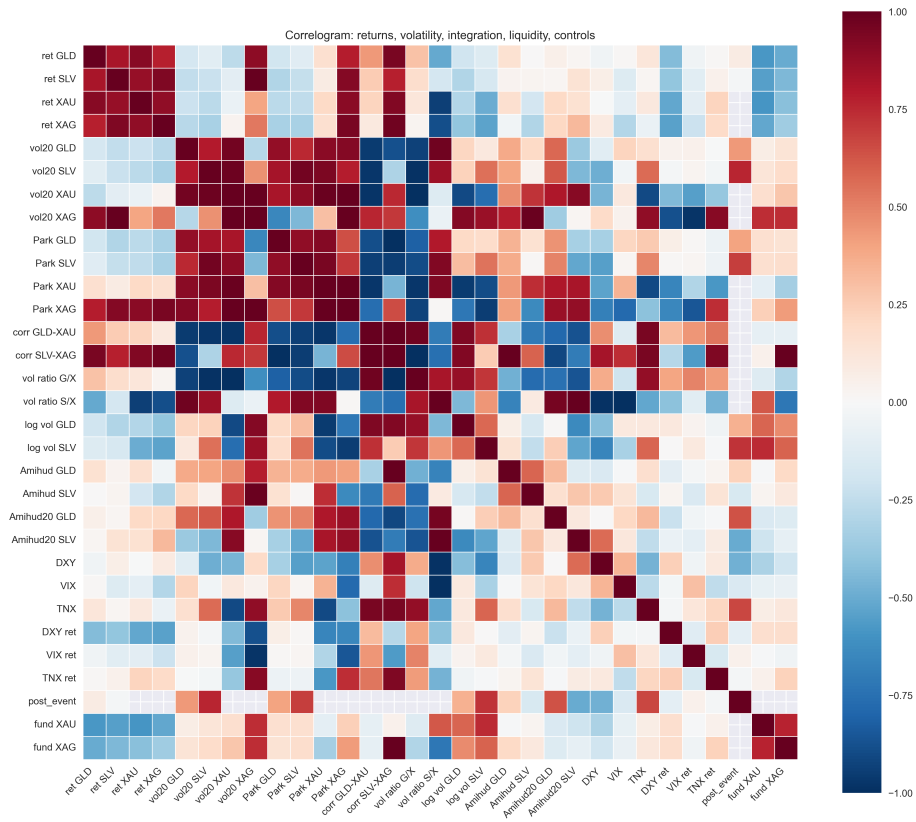
We compute the correlation matrix for a set of variables that are meaningful for the analysis: returns, volatility measures, integration (rolling correlation/beta), liquidity proxies, controls and the event dummy. Only numeric series with

sufficient non-missing overlap are included.

```
# Variables that make sense for correlation (numeric, interpretable)
corr_candidates = [
    "ret_gld", "ret_slv", "ret_xau_binance", "ret_xag_binance",
    "vol_realized_20_ret_gld", "vol_realized_20_ret_slv",
    "vol_realized_20_ret_xau_binance", "vol_realized_20_ret_xag_binance",
    "vol_parkinson_gld", "vol_parkinson_slv",
    "vol_parkinson_xau_binance", "vol_parkinson_xag_binance",
    "corr_20_gld_xau", "corr_20_slv_xag",
    "beta_60_gld_on_xau", "beta_60_slv_on_xag",
    "vol_ratio_gld_xau", "vol_ratio_slv_xag",
    "log_vol_gld", "log_vol_slv",
    "amihud_gld", "amihud_slv", "amihud_20_gld", "amihud_20_slv",
    "dxy", "vix", "tnx", "dxy_ret", "vix_ret", "tnx_ret",
    "post_event", "funding_xau", "funding_xag",
]
corr_cols = [c for c in corr_candidates if c in df_core.columns]
corr_data = df_core[corr_cols].dropna(how="all", axis=1).dropna(how="any", axis=0)
if corr_data.shape[0] < 5:
    corr_data = df_core[corr_cols].dropna(how="all", axis=1)
    corr_data = corr_data.dropna(axis=0, thresh=corr_data.shape[1] // 2)
C = corr_data.corr()
```

```
# Short labels for readability
short = {
    "ret_gld": "ret GLD", "ret_slv": "ret SLV",
    "ret_xau_binance": "ret XAU", "ret_xag_binance": "ret XAG",
    "vol_realized_20_ret_gld": "vol20 GLD", "vol_realized_20_ret_slv": "vol20 SLV",
    "vol_realized_20_ret_xau_binance": "vol20 XAU", "vol_realized_20_ret_xag_binance": "vol2
    "vol_parkinson_gld": "Park GLD", "vol_parkinson_slv": "Park SLV",
    "vol_parkinson_xau_binance": "Park XAU", "vol_parkinson_xag_binance": "Park XAG",
    "corr_20_gld_xau": "corr GLD-XAU", "corr_20_slv_xag": "corr SLV-XAG",
    "beta_60_gld_on_xau": "beta GLD/XAU", "beta_60_slv_on_xag": "beta SLV/XAG",
    "vol_ratio_gld_xau": "vol ratio G/X", "vol_ratio_slv_xag": "vol ratio S/X",
    "log_vol_gld": "log vol GLD", "log_vol_slv": "log vol SLV",
    "amihud_gld": "Amihud GLD", "amihud_slv": "Amihud SLV",
    "amihud_20_gld": "Amihud20 GLD", "amihud_20_slv": "Amihud20 SLV",
    "dxy": "DXY", "vix": "VIX", "tnx": "TNX",
    "dxy_ret": "DXY ret", "vix_ret": "VIX ret", "tnx_ret": "TNX ret",
    "post_event": "post_event", "funding_xau": "fund XAU", "funding_xag": "fund XAG",
}
C_plot = C.rename(index=short, columns=short)
fig, ax = plt.subplots(figsize=(14, 12))
sns.heatmap(
    C_plot,
```

```
    cmap="RdBu_r",
    center=0,
    vmin=-1,
    vmax=1,
    square=True,
    linewidths=0.3,
    ax=ax,
    annot=False,
)
ax.set_title("Correlogram: returns, volatility, integration, liquidity, controls")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```




```
# Summary: variables most correlated with post_event (if any)
if "post_event" in C.columns:
    post_corr = C["post_event"].drop("post_event", errors="ignore")
    post_corr = post_corr.reindex(post_corr.abs().sort_values(ascending=False).index)
    post_corr.head(12)
```

4. Event-study style comparison

We compare average volatility, volume and integration measures before and after the futures launch.

```
def summarize_pre_post(series: pd.Series, label: str) -> pd.DataFrame:
    pre = series[df_core["post_event"] == 0].dropna()
    post = series[df_core["post_event"] == 1].dropna()
    return pd.DataFrame(
        {
            "metric": [label, label],
            "period": ["pre", "post"],
            "mean": [pre.mean(), post.mean()],
            "std": [pre.std(), post.std()],
            "n": [pre.shape[0], post.shape[0]],
        }
    )

rows = []
rows.append(summarize_pre_post(df_core["vol_realized_20_ret_gld"], "GLD 20d vol"))
rows.append(summarize_pre_post(df_core["vol_realized_20_ret_slv"], "SLV 20d vol"))
rows.append(summarize_pre_post(df_core["vol_parkinson_gld"], "GLD Parkinson vol"))
rows.append(summarize_pre_post(df_core["vol_parkinson_slv"], "SLV Parkinson vol"))
if "vol_gld" in df_core.columns:
    rows.append(summarize_pre_post(df_core["vol_gld"], "GLD volume"))
if "vol_slv" in df_core.columns:
    rows.append(summarize_pre_post(df_core["vol_slv"], "SLV volume"))
if "corr_20_gld_xau" in df_core.columns:
    rows.append(summarize_pre_post(df_core["corr_20_gld_xau"], "corr_20_gld_xau"))
if "corr_20_slv_xag" in df_core.columns:
    rows.append(summarize_pre_post(df_core["corr_20_slv_xag"], "corr_20_slv_xag"))

summary_pre_post = pd.concat(rows, ignore_index=True)
summary_pre_post
```

	metric	period	mean	std	n
0	GLD 20d vol	pre	1.343866e-02	6.062648e-03	66
1	GLD 20d vol	post	2.084301e-02	8.844632e-03	26
2	SLV 20d vol	pre	2.411353e-02	6.133201e-03	66
3	SLV 20d vol	post	5.884494e-02	2.324772e-02	26
4	GLD Parkinson vol	pre	8.671084e-03	1.418413e-03	67
5	GLD Parkinson vol	post	1.181838e-02	5.792720e-03	26
6	SLV Parkinson vol	pre	1.625328e-02	2.567734e-03	67
7	SLV Parkinson vol	post	3.233210e-02	1.407079e-02	26
8	GLD volume	pre	1.512372e+07	9.048477e+06	85
9	GLD volume	post	2.534337e+07	1.859671e+07	26

	metric	period	mean	std	n
10	SLV volume	pre	4.388922e+07	2.724932e+07	85
11	SLV volume	post	1.591018e+08	1.001239e+08	26
12	corr_20_gld_xau	pre	NaN	NaN	0
13	corr_20_gld_xau	post	9.213689e-01	1.339276e-02	6
14	corr_20_slv_xag	pre	NaN	NaN	0
15	corr_20_slv_xag	post	9.354342e-01	2.670531e-03	3

For a more formal comparison we can run simple t-tests.

```
from scipy import stats

def ttest_pre_post(series: pd.Series) -> dict:
    pre = series[df_core["post_event"] == 0].dropna()
    post = series[df_core["post_event"] == 1].dropna()
    if len(pre) < 3 or len(post) < 3:
        return {"t_stat": np.nan, "p_value": np.nan}
    t_stat, p_val = stats.ttest_ind(post, pre, equal_var=False)
    return {"t_stat": t_stat, "p_value": p_val}

metrics = {
    "vol_realized_20_ret_gld": "GLD 20d vol",
    "vol_realized_20_ret_slv": "SLV 20d vol",
    "vol_parkinson_gld": "GLD Parkinson vol",
    "vol_parkinson_slv": "SLV Parkinson vol",
    "vol_gld": "GLD volume",
    "vol_slv": "SLV volume",
    "corr_20_gld_xau": "corr_20_gld_xau",
    "corr_20_slv_xag": "corr_20_slv_xag",
}

tt_rows = []
for col, label in metrics.items():
    if col in df_core.columns:
        res = ttest_pre_post(df_core[col])
        tt_rows.append(
            {"metric": label, "t_stat": res["t_stat"], "p_value": res["p_value"]}
        )

tt_results = pd.DataFrame(tt_rows)
tt_results
```

	metric	t_stat	p_value
0	GLD 20d vol	3.921186	3.965328e-04

	metric	t_stat	p_value
1	SLV 20d vol	7.515452	5.090965e-08
2	GLD Parkinson vol	2.738718	1.095236e-02
3	SLV Parkinson vol	5.789406	4.466060e-06
4	GLD volume	2.705857	1.134125e-02
5	SLV volume	5.802084	4.025468e-06
6	corr_20_gld_xau	NaN	NaN
7	corr_20_slv_xag	NaN	NaN

5. Simple regression models

We estimate simple linear models to quantify the impact of the event and controls on ETF volatility and absolute returns.

```
import statsmodels.api as sm

reg_df = df_core.copy()

reg_cols = [
    "vol_realized_20_ret_gld",
    "vol_realized_20_ret_slv",
    "dxy_ret",
    "vix_ret",
    "tnx_ret",
    "post_event",
]

reg_df_reg = reg_df[reg_cols].dropna()
reg_df_reg.head()
```

Date	vol_realized_20_ret_gld	vol_realized_20_ret_slv	dxy_ret	vix_ret	tnx_ret	post
2025-09-30	0.007630	0.013341	-0.001431	0.009877	0.001689	0
2025-10-01	0.007610	0.013404	-0.000614	0.000614	-0.010177	0
2025-10-02	0.007558	0.013260	0.001432	0.020657	-0.004393	0
2025-10-03	0.007311	0.013733	-0.001329	0.001202	0.007555	0
2025-10-06	0.007869	0.013770	0.003983	-0.016960	0.010385	0

```
def run_ols(y_col: str, x_cols: list, data: pd.DataFrame):
    """
    Run a simple OLS regression if there are enough observations.
    Returns a fitted model or None.
    """
```

```
    sub = data[[y_col] + x_cols].dropna()
    if sub.shape[0] < 10:
        print(f"Not enough non-missing observations for {y_col} (n={sub.shape[0]}). Skipping
        return None
    y = sub[y_col]
    X = sm.add_constant(sub[x_cols])
    model = sm.OLS(y, X, missing="drop")
    return model.fit()

model_gld_vol = run_ols(
    "vol_realized_20_ret_gld",
    ["post_event", "dxy_ret", "vix_ret", "tnx_ret"],
    reg_df,
)
model_slv_vol = run_ols(
    "vol_realized_20_ret_slv",
    ["post_event", "dxy_ret", "vix_ret", "tnx_ret"],
    reg_df,
)

if model_gld_vol is not None:
    print(model_gld_vol.summary().tables[1])
    print()
if model_slv_vol is not None:
    print(model_slv_vol.summary().tables[1])
```

```
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.0134      0.001     15.519      0.000       0.012       0.015
post_event     0.0075      0.002      4.600      0.000       0.004       0.011
dxy_ret        0.2238      0.236      0.947      0.346      -0.246       0.694
vix_ret        0.0036      0.009      0.383      0.703      -0.015       0.022
tnx_ret       -0.0053      0.093     -0.057      0.955      -0.189       0.179
==============================================================================

==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.0241      0.002     14.512      0.000       0.021       0.027
post_event     0.0350      0.003     11.148      0.000       0.029       0.041
dxy_ret        0.3997      0.454      0.881      0.381      -0.502       1.301
vix_ret       -0.0017      0.018     -0.095      0.924      -0.037       0.034
tnx_ret       -0.0560      0.178     -0.315      0.753      -0.409       0.297
==============================================================================
```

If needed, we can also model absolute returns as a simple proxy for volatility:

```
if "ret_gld_abs" in reg_df.columns and "ret_slv_abs" in reg_df.columns:
    model_gld_abs = run_ols(
        "ret_gld_abs",
        ["post_event", "dxy_ret", "vix_ret", "tnx_ret"],
        reg_df,
    )
    model_slv_abs = run_ols(
        "ret_slv_abs",
        ["post_event", "dxy_ret", "vix_ret", "tnx_ret"],
        reg_df,
    )

    if model_gld_abs is not None:
        print(model_gld_abs.summary().tables[1])
        print()
    if model_slv_abs is not None:
        print(model_slv_abs.summary().tables[1])
```

```
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.0097      0.002      6.238      0.000       0.007       0.013
post_event     0.0124      0.003      3.842      0.000       0.006       0.019
dxy_ret        0.6160      0.433      1.422      0.158      -0.243       1.475
vix_ret        0.0068      0.018      0.370      0.712      -0.030       0.043
tnx_ret       -0.0439      0.171     -0.257      0.798      -0.383       0.295
==============================================================================

==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.0211      0.004      5.582      0.000       0.014       0.029
post_event     0.0379      0.008      4.838      0.000       0.022       0.053
dxy_ret        2.0522      1.050      1.954      0.053      -0.030       4.134
vix_ret        0.0416      0.045      0.933      0.353      -0.047       0.130
tnx_ret       -0.0396      0.414     -0.095      0.924      -0.861       0.782
==============================================================================
```

6. Interpretation and next steps

- Use the **pre/post summary table** and **t-tests** to decide whether volatility, volume and integration measures changed after the event.

- Use **regression coefficients on `post_event`** to quantify the impact

controlling for DXY, VIX and 10Y yield changes.

- Based on these results, you can write the Empirical Results and Discussion sections of the report, focusing on:
  - sign and significance of the `post_event` coefficient,
  - whether integration metrics (`corr_20`, `beta_60`) increased,
  - whether liquidity improved or worsened.